

Intervenants: Felipe ROCHA et Sara TOUHAMI

Emails: felipe.figuereredo-rocha@u-pec.fr et sara.touhami@u-pec.fr

Méthodes de résolution numérique de problèmes d'analyse

TP1 Introduction Python scientifique

Objectifs du TP

L'objectif de ce TP est de prendre en main les outils fondamentaux de Python scientifique nécessaires à la simulation numérique en mécanique :

- manipuler des tableaux NumPy et des opérations vectorisées ;
- représenter graphiquement des fonctions et des champs numériques ;
- vérifier la convergence d'une approximation numérique ;
- résoudre numériquement des systèmes linéaires simples.

Environnement de travail

On utilisera Python 3 ainsi que les bibliothèques :

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

Le TP peut être réalisé sous VSCode, de préférence en mode Python classique (.py). Une autre option est le mode Jupyter Notebook (fichier .ipynb), qui est toutefois déconseillé au début. Pour tester, ouvrir un nouveau fichier (.py), écrire `print("Hello world")` et l'exécuter avec le bouton *play* (en haut à droite de la fenêtre).

Dans le dépôt github du cours https://github.com/felipefr/numerical_analysis, dans le sous-dossier 2026-2027/tp1, vous trouverez deux fichiers :

`intro_python_basique.ipynb`, `intro_python_scientifique.ipynb`. Ces fichiers contiennent des exemples de syntaxe parmi les plus usuels, pour une référence rapide.

1 Maillage 1D

En simulation numérique, une fonction $u(x)$ est généralement approchée sur un ensemble discret de N points dans l'intervalle $[0, L]$:

$$x_i = i\Delta x, \quad i = 0, \dots, N-1. \quad (1)$$

On va considérer $L = 1$, $N = 21$, et $u(x) = \sin(2\pi x)$. Suivre les étapes ci-dessous :

Travail demandé

1. Initialiser les variables L , N , Δx , $\mathbf{x}$, $\mathbf{u}$ avec des valeurs connues ou avec des zéros.
2. À l'aide d'une boucle `for` avec le constructeur de liste `range`, initialiser chaque composante de $\mathbf{x}$ et $\mathbf{u}$.
3. Tracer $u(x)$ avec le type de ligne `'-o'` et ajouter un titre ainsi que des labels pour les axes.

Voici une liste de fonctions qui pourront vous être utiles :

```
np.zeros, np.sin, np.pi, plt.plot, plt.x(y)label, plt.show, plt.grid
```

2 Opérations vectorisées

On reprend l'exercice précédent en évitant l'utilisation des boucles :

Travail demandé

1. Refaire les étapes de l'exercice 1, mais **sans** utiliser la boucle `for`.
2. Discrétiser $u(x)$ sur un maillage avec le double de subdivisions et tracer la courbe avec `'-x'` au-dessus de la figure précédente, avec les légendes correspondantes.
3. Sauvegarder la figure aux formats `.png` et `.pdf`.

Voici une liste de fonctions qui pourront vous être utiles (en plus de celles de l'exercice précédent)

:

```
np.linspace, np.arange, plt.legend, plt.savefig
```

3 Résolution d'un système linéaire

En mécanique numérique, de nombreux problèmes conduisent à un système linéaire :

$$K\mathbf{u} = \mathbf{f}.$$

On considère :

$$K = \begin{pmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix}, \quad \mathbf{f} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}.$$

Travail demandé

1. Initialiser la matrice K et le vecteur $\mathbf{f}$ sous forme de tableaux NumPy.
2. Vérifier les dimensions de K et $\mathbf{f}$.
3. Résoudre ce système (sans inversion de K) pour trouver $\mathbf{u}$.
4. Vérifier le résidu $\|K\mathbf{u} - \mathbf{f}\|$ pour s'assurer que le système est bien résolu.
5. Calculer K^{-1} .
6. Vérifier que $KK^{-1} = K^{-1}K = I_3$, où I_3 est la matrice identité de dimension 3×3 .
7. Vérifier le résidu $\|\mathbf{u} - K^{-1}\mathbf{f}\|$ pour vérifier que les deux méthodes de résolution du système sont équivalentes.

Voici quelques fonctions utiles :

```
np.array, np.shape, np.linalg.norm, np.linalg.inv
np.linalg.solve(A, b) # résolution du système Ax = b,
A @ x, A @ B # produit matrice-vecteur ou matrice-matrice
```

4 Oscillateur masse–ressort et subplots

On considère l'équation

$$m\ddot{x} + kx = 0, \quad (2)$$

dont la solution analytique pour $x(0) = x_0, \dot{x}(0) = 0$ est :

$$x(t) = x_0 \cos(\omega t), \quad \omega = \sqrt{\frac{k}{m}}. \quad (3)$$

L'énergie mécanique totale de ce système est donnée par :

$$E_m(t) = \frac{1}{2}mv(t)^2 + \frac{1}{2}kx(t)^2, \quad (4)$$

où le premier terme correspond à l'énergie cinétique (E_c) et le deuxième à l'énergie potentielle élastique (E_p).

On prend : $m = 1$ kg, $k = 100$ N/m, $x_0 = 0.01$ m.

Travail demandé

1. Tracer une figure avec des subplots disposés en grille 2×2 , telle que :
 - (a) En haut à gauche : la position $x(t)$.
 - (b) En haut à droite : la vitesse $v(t)$ ($= \dot{x}(t)$).
 - (c) En bas à gauche : l'accélération $a(t)$ ($= \ddot{x}(t)$).
 - (d) En bas à droite : les énergies E_c , E_p et E_m superposées avec les légendes correspondantes.
2. Pouvez-vous constater numériquement la conservation de l'énergie ?

5 Visualisation d'un champ 2D

On considère le champ scalaire $u[0, 1] \times [0, 1] \rightarrow \mathbb{R}$ tel que

$$u(x, y) = \sin(\pi x) \sin(\pi y). \quad (5)$$

On construit un maillage 2D avec :

```

1 x = np.linspace(0, 1, 100)
2 y = np.linspace(0, 1, 100)
3 X, Y = np.meshgrid(x, y)
```

Travail demandé

Représenter le champ avec `contourf()`. Ajouter une barre de couleurs et les noms des axes.

Voici quelques fonctions qui pourront vous être utiles :

```

1 plt.contourf, plt.imshow, plt.colorbar
```

6 Dérivation numérique

On considère la fonction $u(x) = \sin(2\pi x)$, dont la dérivée exacte est $u'(x) = 2\pi \cos(2\pi x)$.

On souhaite approximer cette dérivée à partir des valeurs discrètes u_i , avec trois méthodes :

$$i), u'_A(x_i) \simeq \frac{u_{i+1} - u_i}{\Delta x}, \quad ii), u'_B(x_i) \simeq \frac{u_i - u_{i-1}}{\Delta x}, \quad iii), u'_C(x_i) \simeq \frac{u_{i+1} - u_{i-1}}{2\Delta x}. \quad (6)$$

On va considérer $L = 1$, $N = 101$ (donc $\Delta x = 0.01$).

Travail demandé

1. Quelle est la plage des i valable pour chaque méthode ? (Obs : il y a quelques points extrêmes où il n'est pas possible d'appliquer la ou les formules).
2. Initialiser les variables auxiliaires que vous trouvez pertinentes, en particulier les vecteurs u et du représentant la fonction $u(x)$ et sa dérivée $u'(x)$ discrétisées, respectivement.
3. Calculer les vecteurs représentant la discrétisation de la dérivée de la fonction pour les 3 méthodes, pour les indices possibles. Pour les autres indices, utiliser les valeurs de la dérivée analytique. Vous pouvez éviter l'utilisation des boucles en utilisant les **strides** de **Numpy** (voir conseils ci-dessous).
4. Tracer les 4 courbes sur un même graphique avec les lignes `'-o'`, `'-x'`, `'--'`, et `'-,-'` (analytique).
5. Faites varier N pour avoir plus ou moins de divisions afin de tester si votre code est bien général. Quelle méthode approchée vous semble approcher le mieux la dérivée analytique ?

Voici quelques exemples de **strides** qui pourront vous être utiles :

```

1 v[:-1] # du premier au avant-dernier
2 v[:-2] # du premier au avant-avant-dernier
3 v[1:]  # du deuxieme au dernier
4 v[1:-1] # du deuxieme au avant-dernier
```

7 Étude de convergence

La précision d'une méthode numérique dépend notamment du pas de discrétisation. On reprend l'exercice précédent avec $N \in \{21, 41, 81, 161, 321\}$, et, pour chaque valeur de N , on calcule l'erreur :

$$e_N = \max_i |u'_i - u'_{\text{exact}}(x_i)|.$$

Travail demandé

1. Initialiser les variables auxiliaires que vous trouvez pertinentes, en particulier la liste `N_list = [21, 41, 81, 161, 321]` et les listes vides `erreur_list = []` et `dx_list = []` pour stocker les valeurs des erreurs et des pas de discrétisation, respectivement.
2. Dans une boucle `for`, pour un N donné, calculer le pas de discrétisation et l'erreur pour la méthode i) de l'exercice précédent. Stocker ces valeurs dans les listes correspondantes.
3. Tracer l'erreur en échelle log-log.
4. Refaire ces calculs pour la méthode iii) et ajouter la courbe d'erreur sur le même graphique. Quelle méthode est la plus précise ? Pourquoi ?

Voici quelques bouts de code dont vous pouvez vous servir :

```

1     for N in N_list: # boucle dans une liste
2         np.max, np.linalg.norm, erreur_list.append(e), plt.loglog,
3         plt.xscale('log'), plt.yscale('log')

```

8 Méthode de la bisection

Nous avons vu en cours la méthode de la bisection (voir le fichier 2026-2027/MN_CM1) dans le dépôt git du cours).

Travail demandé

1. Copier-coller le script fourni dans le slide.
2. Modifier la fonction pour retourner l'historique des itérations au lieu de simplement retourner la solution finale.
3. Calculer l'erreur par rapport à la solution finale.
4. Tracer un graphique de convergence de l'erreur (l'ordonnée étant en échelle logarithmique et l'abscisse en échelle linéaire, représentant les itérations).

D'autres références

Pour les connaisseurs de MATLAB :

- <https://personales.unican.es/corcuerp/python/MATLAB%E2%80%93Python%20cheatsheet.html>
- <https://numpy.org/doc/stable/user/numpy-for-matlab-users.html>